

Datasets দিয়ে লোকাল/রিমোট ডেটাসেট লোড করার ধারণা

তুমি হয়তো Hugging Face Hub থেকে `load_dataset()` দিয়ে ডেটাসেট নামাতে জানো। কিন্তু অনেক সময় ডেটা **Hub**-এ থাকে না—বরং:

- তোমার ল্যাপটপে লোকালি আছে (local file)
- অথবা কোনো রিমোট সার্ভার / **GitHub** / **URL**-এ আছে (remote file)

এই ক্ষেত্রে Datasets তোমাকে লোকাল বা রিমোট ফাইল থেকেও ডেটাসেট লোড করতে দেয়।

কোন ফরম্যাটে কীভাবে লোড হয়?

`load_dataset()` এর মধ্যে ২টা জিনিস মেইন:

1. **loading script** (ফরম্যাট বলে দেয়) — যেমন "csv", "json", "text", "pandas"
2. **data_files** (ফাইলের path বা URL)

উদাহরণগুলো:

- **CSV/TSV**
`load_dataset("csv", data_files="my_file.csv")`
- **Text file**
`load_dataset("text", data_files="my_file.txt")`
- **JSON/JSONL**
`load_dataset("json", data_files="my_file.jsonl")`
- **Pickle করা Pandas DataFrame**
`load_dataset("pandas", data_files="my_dataframe.pkl")`

মানে: ডেটা যে ফরম্যাটে আছে, সেই নামটাই প্রথম আর্গুমেন্টে, আর ফাইলটা `data_files` এ।

1. “Loading a local dataset” অংশটা কী বলছে?

এখানে উদাহরণ হিসেবে তারা **SQuAD-it** নামে একটা ডেটাসেট নেবে (ইতালিয়ান Question Answering dataset)।

ডেটাসেটের **train** আর **test** split GitHub-এ আছে, তাই তারা প্রথমে ফাইল ২টা ডাউনলোড করছে `wget` দিয়ে:

```
!wget https://github.com/crux82/squad-it/raw/master/SQuAD_it-train.json.gz
!wget https://github.com/crux82/squad-it/raw/master/SQuAD_it-test.json.gz
```

এখানে `.json.gz` মানে:

- ভেতরে JSON আছে
- কিন্তু gzip দিয়ে কমপ্রেস করা

এরপর সাধারণত কীভাবে **datasets** দিয়ে লোড করবে?

ডাউনলোড হয়ে গেলে একই ফোল্ডারে থাকলে এভাবে লোড করা হয়:

```
from datasets import load_dataset

data_files = {
    "train": "SQuAD_it-train.json.gz",
    "test": "SQuAD_it-test.json.gz"
}

dataset = load_dataset("json", data_files=data_files)
```

এখানে:

- `"json"` বলছে ডেটা JSON ফরম্যাটের
- `data_files` দিয়ে train/test আলাদা করে দেওয়া হয়েছে
- `dataset["train"]`, `dataset["test"]` দিয়ে স্প্লিটগুলো পাওয়া যাবে

দ্রুত চেক করার জন্য

```
print(dataset)
print(dataset["train"][0])
```

1) JSON বনাম JSON Lines (JSONL) — পার্থক্যটা কী?

Ordinary JSON (nested/dictionary টাইপ)

একটা ফাইলের ভেতরে একটাই বড় **JSON** অবজেক্ট/লিস্ট থাকে, ভেতরে অনেক nested structure থাকতে পারে।

JSON Lines (JSONL)

ফাইলের প্রতি লাইনে একটা করে **JSON object** থাকে।
মানে line-separated JSON।

SQuAD-it ডেটাসেটটা **ordinary nested JSON** ফরম্যাট ব্যবহার করে এবং আসল কনটেন্টটা **data** নামের ফিল্ডে রাখা।

2) **field="data"** কেন দরকার?

কারণ JSON ফাইলটা এমন হতে পারে:

```
{  
  "version": "...",  
  "data": [ ... আসল উদাহরণগুলো এখানে ... ]  
}
```

Datasets কে বলতে হয়—“ডেটা **data** ফিল্ডের ভেতরে”
তাই:

```
from datasets import load_dataset  
squad_it_dataset = load_dataset("json", data_files="SQuAD_it-train.json", field="data")
```

3) কেন **DatasetDict** আসে এবং কেন **default split = train**?

লোকাল ফাইল থেকে লোড করলে **load_dataset()** ডিফল্টভাবে ধরে নেয় এটা **train split**।
তাই তুমি দেখবে:

```
DatasetDict({  
  train: Dataset(...)  
})
```

এখানে:

- **DatasetDict** = কয়েকটা split একসাথে রাখা container

- `train` = একটাই split আছে (কারণ তুমি একটাই ফাইল দিয়েছ)

4) একটা **example** কীভাবে দেখা হয়?

```
squad_it_dataset["train"][0]
```

এতে তুমি প্রথম row/entry পাবে।

ওই entry তে দেখা যাচ্ছে:

- `title`: আর্টিকেলের/টপিকের নাম
- `paragraphs`: অনেকগুলো paragraph
 - প্রতিটায় `context` (প্যাসেজ টেক্সট)
 - `qas` (question-answer সেট)
 - `question`
 - `answers` (answer text + শুরু index `answer_start`)
 - `id`

অর্থাৎ এটা **QA dataset**—একটা context থেকে প্রশ্ন-উত্তর।

5) Train + Test একসাথে কেন দরকার?

কারণ তুমি যদি preprocessing করতে চাও (যেমন `map()` দিয়ে tokenize/clean করা), তাহলে আলাদা আলাদা split এ না করে একসাথে করতে সুবিধা:

একবারেই train এবং test দুইটায় একই প্রসেস চালানো যায়।

6) Train + Test একসাথে লোড করার নিয়ম

`data_files` এ dictionary দাও, যেখানে split নাম → ফাইল path:

```
data_files = {"train": "SQuAD_it-train.json", "test": "SQuAD_it-test.json"}
squad_it_dataset = load_dataset("json", data_files=data_files, field="data")
```

ফলে আউটপুট হবে:

```
DatasetDict({
  train: Dataset(num_rows=442)
  test: Dataset(num_rows=48)
})
```

7) **data_files** কতটা **flexible**?

data_files হতে পারে:

- একটা **string path**
`"file.json"`
- একটা **list**
`["a.json", "b.json"]`
- একটা **dict (split mapping)**
`{"train": "...", "test": "..."}"`

এছাড়া glob pattern (Unix shell style) ব্যবহার করা যায়:

```
data_files="*.json"
```

মানে ওই ফোল্ডারের সব `.json` একসাথে লোড হবে (একটা split হিসেবে)।

8) **.gz** ফাইল **decompress** না করেও লোড করা যায়

Datasets **automatic decompression** করে দেয় (GZIP/ZIP/TAR ইত্যাদি)।

তাই এটা valid:

```
data_files = {"train": "SQuAD_it-train.json.gz", "test": "SQuAD_it-test.json.gz"}
squad_it_dataset = load_dataset("json", data_files=data_files, field="data")
```

এতে আলাদা করে `gzip -d` দিয়ে unzip করতে হবে

2. রিমোট ডেটাসেট লোড করা (**Remote dataset**)

কোম্পানিতে কাজ করলে ডেটা অনেক সময় লোকাল মেশিনে না থেকে থাকে:

- GitHub / S3 / কোনো HTTP server
- কোম্পানির internal server (যেখানে URL দিয়ে ফাইল অ্যাক্সেস করা যায়)

Datasets-এর সুবিধা হলো: লোকাল ফাইলের মতোই রিমোট ফাইলও লোড করা যায়—শুধু `data_files` এ লোকাল path না দিয়ে **URL** দিতে হয়।

মূল আইডিয়া

- লোকাল হলে: `data_files="SQuAD_it-train.json.gz"`
- রিমোট হলে: `data_files="https://.../SQuAD_it-train.json.gz"`

Datasets নিজে থেকে:

- URL থেকে ফাইল ডাউনলোড করবে (cache-এ রাখে)
- `.gz` হলে **automatic decompression** করবে
- তারপর Dataset তৈরি করবে

SQuAD-it (GitHub) রিমোট থেকে লোড করার উদাহরণ

```
from datasets import load_dataset
```

```
url = "https://github.com/crux82/squad-it/raw/master/"
data_files = {
    "train": url + "SQuAD_it-train.json.gz",
    "test": url + "SQuAD_it-test.json.gz",
}
```

```
squad_it_dataset = load_dataset("json", data_files=data_files, field="data")
```

এখানে কী হচ্ছে?

- `url` হলো base URL
- `data_files` dict দিয়ে split mapping করা হয়েছে (`train`, `test`)
- `field="data"` কারণ nested JSON-এর আসল ডেটা `data` ফিল্ডে
- `"json"` loader ব্যবহার করা হয়েছে (ফাইল JSON ফরম্যাটে)